SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192525458
Student Name	S. Sharvesh

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

Comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Criteria	**Weightage (%)**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**

Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I S.Sharvesh(192525458)_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Sharvesh_____ Date: 12-10-2026_____

SCALABILITY AUDIT OF AN E-COMMERCE PLATFORM

(a) UNDERSTAND THE PROBLEM

1. Introduction

ShopSphere is an online shopping platform whose product catalogue currently contains 100,000 products and is expected to grow to 1,000,000 products within one year. The platform consists of five backend modules: Duplicate-Listing Detector, API Rate Limiter, Product Search, Recommendation Engine, and Sales Analytics Engine.

As the input size increases, the number of operations performed by an algorithm also increases. Therefore, it is necessary to analyse the time complexity and space complexity of each module to determine whether it can handle the growing catalogue efficiently.

The objective of this scalability audit is to identify the dominant basic operation of each module, calculate its operation count, determine its asymptotic complexity, apply mathematical techniques such as backward substitution and the Master Theorem, and recommend suitable redesigns.

The server executes 10^8 basic operations per second. Therefore,

Execution time = Number of basic operations / 10^8

The nightly processing window is 10 minutes.

Maximum operations allowed = $10 \times 60 \times 10^8$

Maximum operations allowed = 60,000,000,000 operations.

Any module requiring more than 60 billion basic operations will exceed this theoretical processing budget when considered individually.

2. Identification of Input Size and Basic Operations

Module M1 – Duplicate-Listing Detector

- Input size: n products.
- Basic operation: Comparing product titles and seller identifiers.
- Growth: $\Theta(n^2)$.
- Justification: Every product is compared with every product appearing after it, resulting in a quadratic number of comparisons.

Module M2 – API Rate Limiter

- Input size: n requests.
- Basic operation: Auditing an individual request.
- Growth: $\Theta(n)$.

- Justification: The inner loop executes 1, 2, 4, 8, and subsequent powers of two. The sum of these operations forms a geometric series.

Module M3 – Product Search S1

- Input size: n products.
- Basic operation: Comparing the search key with each product.
- Growth: Best case $\Theta(1)$; average and worst cases $\Theta(n)$.
- Justification: Sequential search checks products one after another until the target is found or the list ends.

Module M3 – Product Search S2

- Input size: n entries in a sorted product index.
- Basic operation: Comparing the search key with the middle entry.
- Growth: $\Theta(\log n)$.
- Justification: Each comparison eliminates approximately half of the remaining search space.

Module M4 – Recommendation Engine

- Current design: $T(n) = 4T(n/2) + n$, with complexity $\Theta(n^2)$.
- Proposed design: $T(n) = 2T(n/2) + n$, with complexity $\Theta(n \log n)$.
- Justification: The current design creates four recursive subproblems, whereas the proposed design removes replication and creates two.

Module M5 – Sales Analytics Engine

- Input size: n sales records.
- Basic operation: Processing and sorting records during recursive execution.
- Recurrence: $T(n) = 2T(n/2) + n \log n$.
- Growth: $\Theta(n \log^2 n)$.
- Justification: The algorithm divides the data into halves and performs n log n work at each recursive level.

(b) ANALYSE THE ITERATIVE MODULES

1. M1 – Duplicate-Listing Detector

Basic Operation

The basic operation is the comparison:

Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller

The outer loop selects a product, and the inner loop compares it with every later product.

For each value of i, the inner loop executes n − i − 1 times.

Derivation of the Exact Number of Comparisons

Let C(n) denote the total number of comparisons.

$C(n) = \Sigma(i = 0 \text{ to } n - 2) (n - i - 1)$

Expanding,

$C(n) = (n - 1) + (n - 2) + ... + 2 + 1$

Using the formula for the sum of the first n − 1 positive integers,

$1 + 2 + ... + (n - 1) = n(n - 1)/2$

Therefore,

$C(n) = n(n - 1)/2$

Expanding,

$C(n) = (n^2 - n)/2$

$C(n) = n^2/2 - n/2$

The dominant term is $n^2/2$. Hence, the exact comparison count grows quadratically.

Time Complexity:

$T(n) = \Theta(n^2)$

Space Complexity:

The algorithm uses only loop variables and does not require an additional data structure proportional to n.

$S(n) = \Theta(1)$, excluding the existing product catalogue and any separately maintained duplicate records.

Numerical Calculation for 100,000 Products

Given n = 100,000,

C(n) = 100,000 × 99,999 / 2

C(n) = 4,999,950,000 comparisons.

Execution time = $4,999,950,000 / 10^8$

Execution time ≈ 49.9995 seconds.

Numerical Calculation for 1,000,000 Products

Given n = 1,000,000,

C(n) = 1,000,000 × 999,999 / 2

C(n) = 499,999,500,000 comparisons.

Execution time = 499,999,500,000 / 10^8

Execution time = 4,999.995 seconds.

Execution time in minutes = 4,999.995 / 60

Execution time ≈ 83.33 minutes.

Interpretation: M1 exceeds the 10-minute processing window when the catalogue reaches one million products. Therefore, it must be redesigned using a more efficient duplicate-detection method.

A suitable solution is to use a hash table with a composite key containing the normalized product title and seller ID. This can provide expected Θ(n) processing under suitable hashing assumptions.

2. M2 – API Rate Limiter

Given Algorithm

The variable i starts at 1 and doubles after every iteration. The inner loop executes i audit operations.

The sequence of audit counts is:

1, 2, 4, 8, 16, 32, ...

Derivation of the Exact Number of Audit Operations

For n > 1, let k = ceil($\log_2$ n).

The last executed value of i is 2^(k − 1).

Therefore,

A(n) = 1 + 2 + 4 + ... + 2^(k − 1)

Using the geometric-series formula,

A(n) = 2^k − 1

Substituting k,

A(n) = 2^ceil($\log_2$ n) − 1

This is the exact audit count for integer n > 1. If n is a power of two, the expression simplifies to n − 1. When n = 1, the loop performs zero audits.

Determine the Tight Asymptotic Bound

For n > 1,

n ≤ 2^ceil($\log_2$ n) < 2n

Therefore,

n − 1 ≤ A(n) < 2n − 1

The operation count is bounded above and below by constant multiples of n.

Hence,

Time Complexity: Θ(n)

Space Complexity: Θ(1)

Verify the Result for n = 1,024

Given,

$n = 1,024 = 2^{10}$

The executed values are:

1, 2, 4, 8, 16, 32, 64, 128, 256, 512.

Therefore,

$A(1,024) = 1 + 2 + 4 + ... + 512$

$A(1,024) = 2^{10} - 1$

$A(1,024) = 1,024 - 1$

A(1,024) = 1,023 audit operations.

Evaluation of the Team Member's Claim

The team member claims that the complexity is Θ(n log n). This claim is incorrect.

Although the outer loop executes logarithmically many iterations, the inner loop does not execute n times during every iteration. Instead, the number of audits increases geometrically.

The total number of audits is less than 2n.

Therefore, the correct tight bound is:

A(n) = Θ(n).

M2 is efficient under the given model and does not require an asymptotic redesign.

(c) SOLVE A RECURRENCE BY BACKWARD SUBSTITUTION

M3 – Product Search S2

The recurrence supplied in the question is:

$T(n) = T(n/2) + 1$

Base condition:

$T(1) = 1$

Here, T(n) represents the number of comparisons. At every step, the algorithm makes one comparison and reduces the remaining search space by half.

Step 1: First Substitution

$T(n) = T(n/2) + 1$

Expanding $T(n/2)$,

$T(n/2) = T(n/4) + 1$

Substituting,

$T(n) = T(n/4) + 1 + 1$

$T(n) = T(n/4) + 2$

Step 2: Second Substitution

$T(n/4) = T(n/8) + 1$

Therefore,

$T(n) = T(n/8) + 3$

Step 3: Third Substitution

$T(n) = T(n/16) + 4$

Continuing the same process, after k substitutions,

$T(n) = T(n/2^k) + k$

Step 4: Determine the Number of Substitutions

The recurrence reaches the base case when the input size becomes one.

$n/2^k = 1$

Therefore,

$n = 2^k$

Taking logarithm base 2,

$k = \log_2 n$

Step 5: Obtain the Exact Closed Form

Substitute $k = \log_2 n$ into the expanded recurrence.

$T(n) = T(n/2^k) + k$

$T(n) = T(1) + \log_2 n$

Since $T(1) = 1$,

$T(n) = \log_2 n + 1$.

This exact expression applies when n is a power of two under the recurrence given in the question. For arbitrary input sizes, integer rounding affects the precise count, but not the asymptotic result.

Time Complexity

$T(n) = \Theta(\log n)$

The algorithm is highly efficient because each comparison eliminates approximately half of the remaining search entries.

For example, for n = 1,024,

$T(n) = \log_2 1{,}024 + 1$

$T(n) = 10 + 1$

$T(n) = 11$ comparisons under the specified recurrence.

Space Complexity of Iterative and Recursive Implementations

Iterative Binary Search

The iterative implementation uses a constant number of variables, such as low, high and mid.

Auxiliary space = $\Theta(1)$.

Time complexity = $\Theta(\log n)$.

Recursive Binary Search

The recursive implementation creates a new function call at every halving step. Its maximum recursion depth is logarithmic.

Auxiliary space = $\Theta(\log n)$.

Time complexity = $\Theta(\log n)$.

Conclusion: Both implementations have logarithmic running time. The iterative implementation uses less auxiliary memory and is suitable when minimizing stack usage is important.

(d) APPLY THE MASTER THEOREM

The Master Theorem solves divide-and-conquer recurrences of the form:

$T(n) = aT(n/b) + f(n)$

Here:

- a is the number of recursive subproblems.
- b is the factor by which the input size is reduced.
- f(n) is the non-recursive work.
- n^(log_b a) is the benchmark used to identify the applicable case.

1. M4 – Current Recommendation Engine

Given recurrence:

$T(n) = 4T(n/2) + n$

Step 1: Identify the Parameters

Comparing with $T(n) = aT(n/b) + f(n)$,

$a = 4$

$b = 2$

$f(n) = n$

Step 2: Calculate the Benchmark Function

$n^{(\log_b a)} = n^{(\log_2 4)}$

Since $\log_2 4 = 2$,

$n^{(\log_b a)} = n^2$

Step 3: Compare f(n) with the Benchmark

$f(n) = n$

Benchmark = n^2

The function n is polynomially smaller than n^2 because $n = O(n^{(2 - 1)})$.

Step 4: Apply the Master Theorem

Master Theorem Case 1 applies.

Therefore,

$T(n) = \Theta(n^2)$.

Interpretation

The current recommendation engine creates four recursive subproblems of size n/2. The replicated partitions significantly increase the amount of recursive work.

The quadratic complexity makes the current design expensive for large customer-product datasets.

2. M4 – Proposed Recommendation Engine

The proposed recurrence is:

$T(n) = 2T(n/2) + n$

Step 1: Identify the Parameters

$a = 2$

$b = 2$

$f(n) = n$

Step 2: Calculate the Benchmark Function

$n^{(\log_b a)} = n^{(\log_2 2)}$

Since $\log_2 2 = 1$,

$n^{(\log_b a)} = n$

Step 3: Compare the Functions

$f(n) = n$

Benchmark = n

Therefore,

$f(n) = \Theta(n^{(\log_b a)})$

Step 4: Apply the Master Theorem

Master Theorem Case 2 applies with logarithmic power zero.

Therefore,

$T(n) = \Theta(n \log n)$.

Interpretation

Removing replication reduces the number of recursive subproblems from four to two. The time complexity improves from $\Theta(n^2)$ to $\Theta(n \log n)$.

The proposed design is significantly more scalable.

3. Effect of Doubling the Input Size

Current Design

$T(n) = \Theta(n^2)$

When the input doubles,

$T(2n) = \Theta((2n)^2)$

$T(2n) = \Theta(4n^2)$

Therefore, the running time increases approximately fourfold.

Proposed Design

$T(n) = \Theta(n \log_2 n)$

When the input doubles,

$T(2n) = \Theta(2n \log_2(2n))$

Using $\log_2(2n) = \log_2 n + 1$,

$T(2n) = \Theta(2n(\log_2 n + 1))$

The approximate ratio is:

$T(2n)/T(n) \approx 2 + 2/\log_2 n$

For large n, this ratio approaches two.

Conclusion: The proposed recommendation engine scales much better than the current design.

4. M5 – Sales Analytics Engine

Given recurrence:

$T(n) = 2T(n/2) + n \log n$

Step 1: Identify the Parameters

a = 2

b = 2

f(n) = n log n

Step 2: Calculate the Benchmark Function

n^(log_b a) = n^($\log_2$ 2)

n^(log_b a) = n

Step 3: Determine Whether a Standard Master Theorem Case Applies

The benchmark is n, while the non-recursive work is n log n.

- Case 1 does not apply because f(n) is not polynomially smaller than n.
- The basic form of Case 2 does not directly apply because f(n) is not Θ(n).
- Case 3 does not apply because f(n) is not polynomially larger than n.

Therefore, the extended Master Theorem or a recursion-tree method is required.

Step 4: Apply the Extended Master Theorem

The extended Master Theorem states that when

T(n) = aT(n/b) + Θ(n^(log_b a) $\log^k$ n),

the solution is

T(n) = Θ(n^(log_b a) log^(k+1) n).

For M5,

n^(log_b a) = n

and k = 1.

Therefore,

$T(n) = \Theta(n \log^2 n)$

Final Complexity: $\Theta(n \log^2 n)$.

Verification Using a Recursion Tree

At level 0, the work is:

$n \log_2 n$

At level 1, there are two subproblems of size n/2:

$2 \times (n/2) \log_2(n/2) = n(\log_2 n - 1)$

At level 2, there are four subproblems of size n/4:

$4 \times (n/4) \log_2(n/4) = n(\log_2 n - 2)$

At level i, the total work is:

$n(\log_2 n - i)$

Let $k = \log_2 n$. Summing the internal levels gives:

$T(n) = n[k + (k - 1) + ... + 1]$

Using the arithmetic-series formula,

$T(n) = nk(k + 1)/2$

Substituting $k = \log_2 n$,

$T(n) = n \log_2 n(\log_2 n + 1)/2$

The dominant term is proportional to $n \log^2 n$.

Therefore,

$T(n) = \Theta(n \log^2 n)$.

(e) ANALYSE BEST, AVERAGE AND WORST CASES

1. S1 – Sequential Search

Sequential search examines the product catalogue from the first entry onwards.

Best Case

Condition: The searched product is the first entry.

Number of comparisons = 1.

Time complexity = $\Theta(1)$.

Average Case

Condition: The product exists and is equally likely to appear at any of the n positions.

The comparison counts are:

1, 2, 3, ..., n.

Average number of comparisons:

Cavg = (1 + 2 + 3 + ... + n)/n

Using the arithmetic-series formula,

Cavg = [n(n + 1)/2]/n

Cavg = (n + 1)/2.

Time complexity = Θ(n).

Worst Case

Condition: The product is at the last position or is absent from the catalogue.

Number of comparisons = n.

Time complexity = Θ(n).

Summary of S1

Best case: 1 comparison, Θ(1).

Average case when present: (n + 1)/2 comparisons, Θ(n).

Worst case: n comparisons, Θ(n).

2. Expected Comparisons for S1 When the Product Exists with Probability p

Let p be the probability that the product exists.

Probability that the product does not exist = 1 − p.

If the product exists, the expected comparison count is:

Cpresent = (n + 1)/2

If the product does not exist, all n entries are checked:

Cabsent = n

Using the weighted average,

E[C] = pCpresent + (1 − p)Cabsent

Substituting,

E[C] = p((n + 1)/2) + (1 − p)n

Expanding,

$E[C] = p(n + 1)/2 + n - pn$

Combining terms,

$E[C] = n - p(n - 1)/2$.

Numerical Evaluation

Given:

$n = 1{,}000{,}000$

$p = 0.5$

Substituting,

$E[C] = 1{,}000{,}000 - 0.5(999{,}999)/2$

$E[C] = 1{,}000{,}000 - 249{,}999.75$

$E[C] = 750{,}000.25$ comparisons.

The expected number of comparisons is approximately 750,000 per search.

Estimated execution time:

$t = 750{,}000.25 / 10^8$

$t \approx 0.0075$ seconds.

This is an idealized estimate. Actual response time can be higher because of database operations, network delays and other system overheads.

3. S2 – Binary Search

Binary search works on a sorted product index and eliminates half the remaining entries after each comparison.

Best Case

Condition: The searched product is exactly at the middle position in the first comparison.

Number of comparisons = 1.

Time complexity = $\Theta(1)$.

Average Case

Condition: The target is located at a typical depth in a balanced binary-search process.

The average number of comparisons grows logarithmically.

Time complexity = $\Theta(\log n)$.

Worst Case

Condition: The target is on the deepest search path, or the product is absent and the algorithm continues until the search interval becomes empty.

The remaining search space is halved at each step.

Time complexity = Θ(log n).

The exact comparison count depends on the implementation and the target position. The recurrence supplied in the question gives $\log_2 n + 1$ comparisons for powers of two when T(1) = 1 is included.

Comparison Between S1 and S2

Sequential search works without sorting but may examine every product. Binary search requires a sorted index but reduces the number of comparisons to logarithmic growth.

Therefore, S2 is more suitable for frequent product lookups in ShopSphere, provided the sorted index is maintained correctly.

(f) PROVE AN ASYMPTOTIC BOUND

The report generator performs:

$f(n) = 3n^2 + 10n \log_2 n + 50$

We must prove that $f(n) = \Theta(n^2)$ and that $f(n)$ is not $O(n)$.

1. Formal Definition of Big-Theta

A function f(n) is Θ(g(n)) if positive constants c_1 and c_2 and an integer n_0 exist such that:

$0 \le c_1 g(n) \le f(n) \le c_2 g(n)$

for every $n \ge n_0$.

Here, $g(n) = n^2$.

Therefore, we need to establish:

$c_1 n^2 \le 3n^2 + 10n \log_2 n + 50 \le c_2 n^2$.

2. Establish the Lower Bound

Given,

$f(n) = 3n^2 + 10n \log_2 n + 50$

For every integer $n \ge 2$, the terms $10n \log_2 n$ and 50 are non-negative.

Therefore,

$f(n) \ge 3n^2$.

Choose:

$c_1 = 3$.

Thus,

$c_1 n^2 \le f(n)$.

The lower bound is established.

3. Establish the Upper Bound

For every integer $n \geq 2$,

$\log_2 n \leq n$.

Multiplying by n,

$n \log_2 n \leq n^2$.

Multiplying by 10,

$10n \log_2 n \leq 10n^2$.

Also, for $n \geq 2$,

$50 \leq 50n^2$.

Combining these inequalities,

$f(n) = 3n^2 + 10n \log_2 n + 50$

$f(n) \leq 3n^2 + 10n^2 + 50n^2$

$f(n) \leq 63n^2$.

Choose:

$c_2 = 63$.

Therefore,

$f(n) \leq c_2 n^2$.

4. Select the Threshold

Both bounds hold for every integer $n \geq 2$.

Choose:

$n_0 = 2$.

Thus, for every $n \geq 2$,

$3n^2 \leq 3n^2 + 10n \log_2 n + 50 \leq 63n^2$.

Therefore, the constants are:

$c_1 = 3$

$c_2 = 63$

$n_0 = 2$

These constants satisfy the formal definition.

Hence, $f(n) = \Theta(n^2)$.

The chosen constants are valid and do not need to be the tightest possible constants.

5. Prove That f(n) Is Not O(n)

By definition, f(n) is O(n) if a constant c > 0 and a threshold n_0 exist such that:

$f(n) \leq cn$

for every $n \geq n_0$.

However, we have already established that:

$f(n) \geq 3n^2$.

Dividing by n,

$f(n)/n \geq 3n$.

As n approaches infinity,

3n approaches infinity.

Therefore,

$\lim(n \to \infty) f(n)/n = \infty$.

If f(n) were O(n), this ratio would remain bounded above by a constant for sufficiently large n. Since it grows without bound, no such constant can exist.

Hence, f(n) is not O(n).

The report generator has quadratic growth and cannot be bounded by a constant multiple of a linear function.

(g) SCALABILITY DECISION

1. Processing Budget

The server executes 10^8 basic operations per second.

The nightly processing window is 10 minutes.

Total available time = 10 × 60 = 600 seconds.

Maximum operation budget:

$600 \times 10^8 = 60{,}000{,}000{,}000$ operations.

The following estimates use exact operation counts where derived and dominant-term estimates for recurrences where the exact count cannot be determined without further implementation details.

2. M1 – Duplicate-Listing Detector

The exact operation count is:

$C(n) = n(n - 1)/2$.

For n = 1,000,000,

C(n) = 499,999,500,000 comparisons.

Execution time:

t = 499,999,500,000 / 10^8

t = 4,999.995 seconds.

In minutes,

t ≈ 83.33 minutes.

Decision: M1 fails the 10-minute processing window and must be redesigned.

Recommendation: Replace pairwise comparisons with hash-based duplicate detection using normalized product titles and seller IDs as a composite key. This can achieve expected Θ(n) processing under suitable hashing assumptions.

3. M2 – API Rate Limiter

The exact audit count is:

A(n) = 2^ceil($\log_2$ n) − 1.

For n = 1,000,000,

ceil($\log_2$ 1,000,000) = 20.

Therefore,

A(n) = 2^{20} − 1

A(n) = 1,048,575 audits.

Execution time:

t = 1,048,575 / 10^8

t ≈ 0.01049 seconds.

Decision: M2 satisfies the processing window under the given model. No asymptotic redesign is required.

4. M3 – Product Search S2

The recurrence is:

T(n) = T(n/2) + 1.

For powers of two, the closed form is:

T(n) = $\log_2$ n + 1.

For n = 1,000,000,

T(n) ≈ $\log_2$ 1,000,000 + 1

$T(n) \approx 19.93 + 1$

$T(n) \approx 20.93$ comparisons under the recurrence-based estimate.

Execution time:

$t \approx 20.93 / 10^8$

$t \approx 2.09 \times 10^{-7}$ seconds.

This is a theoretical estimate, not a prediction of actual end-to-end search latency.

Decision: S2 easily satisfies the processing requirement. Maintain the sorted index and prefer iterative binary search when constant auxiliary space is important.

5. M4 – Current Recommendation Engine

The current recurrence is:

$T(n) = 4T(n/2) + n$.

Its complexity is:

$T(n) = \Theta(n^2)$.

Using n^2 as a dominant-term benchmark,

$T(n) \approx (1{,}000{,}000)^2$

$T(n) \approx 1{,}000{,}000{,}000{,}000$ operations.

Estimated execution time:

$t \approx 10^{12} / 10^8$

$t \approx 10{,}000$ seconds.

In minutes,

$t \approx 166.67$ minutes.

This is an illustrative asymptotic benchmark, not an exact operation count from the recurrence.

Decision: The current design fails the 10-minute target under this benchmark and must be redesigned.

Recommendation: Remove partition replication and use the proposed two-subproblem design.

6. M4 – Proposed Recommendation Engine

The proposed recurrence is:

$T(n) = 2T(n/2) + n$.

Its complexity is:

$T(n) = \Theta(n \log_2 n)$.

Using $n \log_2 n$ as a dominant-term benchmark,

$T(n) \approx 1{,}000{,}000 \times \log_2 1{,}000{,}000$

$T(n) \approx 1{,}000{,}000 \times 19.9316$

$T(n) \approx 19{,}931{,}569$ operations.

Estimated execution time:

$t \approx 19{,}931{,}569 / 10^8$

$t \approx 0.1993$ seconds.

This is an asymptotic benchmark estimate; actual execution depends on the implementation and its constant factors.

Decision: The proposed design is much more scalable and fits the processing window under the stated model.

7. M5 – Sales Analytics Engine

The recurrence is:

$T(n) = 2T(n/2) + n \log_2 n$.

Its complexity is:

$T(n) = \Theta(n \log^2 n)$.

Using $n \log_2^2 n$ as a dominant-term benchmark,

$T(n) \approx 1{,}000{,}000 \times (19.9316)^2$

$T(n) \approx 397{,}266{,}000$ operations.

Estimated execution time:

$t \approx 397{,}266{,}000 / 10^8$

$t \approx 3.97$ seconds.

This estimate uses the dominant asymptotic term and is not an exact operation count.

Decision: M5 is expected to fit within the 10-minute processing window under the stated model. Nevertheless, actual execution should be measured because sorting, memory allocation, data movement and storage access can affect performance.

8. Consolidated Scalability Results

1. M1: Approximately 499,999,500,000 comparisons; estimated time 83.33 minutes; fails the deadline.
2. M2: Exactly 1,048,575 audits; estimated time 0.01049 seconds; passes.
3. M3 S2: Approximately 20 comparisons; theoretical time around 2.1×10^{-7} seconds under the given model; passes.
4. M4 current design: Quadratic complexity; n^2 benchmark of 10^{12} operations; estimated time 166.67 minutes; fails under the benchmark.

5. M4 proposed design: Linearithmic complexity; n $\log_2$ n benchmark of approximately 19.93 million operations; estimated time 0.199 seconds; passes under the benchmark.

6. M5: Θ(n log² n); dominant-term benchmark of approximately 397.27 million operations; estimated time 3.97 seconds; passes under the benchmark.

The M1 and M2 counts follow their exact derived formulas. S2 is an approximate comparison estimate. M4 and M5 use dominant-term benchmarks because their recurrences do not specify all implementation constants and base-case costs. Their actual completion times must be confirmed through profiling.

9. Final Recommendations

Recommendation 1: Redesign M1.

M1 should be prioritized because its quadratic comparison count exceeds the processing window. Hash-based grouping by normalized title and seller ID can reduce expected processing to Θ(n), subject to appropriate hashing and collision handling.

Recommendation 2: Redesign M4.

Remove replicated partitions and use the proposed recurrence T(n) = 2T(n/2) + n. This improves the complexity from Θ(n²) to Θ(n log n).

Recommendation 3: Retain M2.

M2 is Θ(n), not Θ(n log n). The existing geometric-loop structure is efficient under the given operation model. Production traffic and concurrency should still be tested.

Recommendation 4: Retain M3 S2.

Binary search provides logarithmic search time. Maintaining a sorted index enables efficient product lookups, and an iterative implementation uses constant auxiliary space.

Recommendation 5: Profile M5.

M5 has Θ(n log² n) complexity and appears manageable under the dominant-term estimate. However, it should be tested with realistic sales records and production-like memory and storage conditions.

Recommendation 6: Introduce scalability testing.

Run automated tests with increasing input sizes and measure execution time, peak memory usage and nightly job completion time.

Recommendation 7: Evaluate the complete nightly workload.

The modules may run sequentially or compete for shared server resources. The total nightly duration must therefore be measured rather than assuming that individual module timings represent the full workload.

OVERALL CONCLUSION

The scalability audit demonstrates the importance of analysing algorithm growth before expanding ShopSphere's catalogue.

M1 performs n(n − 1)/2 comparisons and has quadratic complexity. At one million products, it requires approximately 83.33 minutes under the stated operation-rate model, exceeding the 10-minute processing window. Therefore, its pairwise comparison method must be redesigned.

M2 performs a geometric sum of audit operations and has a tight linear bound of $\Theta(n)$. The team member's $\Theta(n \log n)$ claim is incorrect.

M3 S2 achieves logarithmic search time and is suitable for frequent product lookups when a sorted index is maintained. Iterative binary search also provides constant auxiliary space.

M4's current design has quadratic complexity because of replicated recursive partitions. The proposed redesign reduces the complexity to $\Theta(n \log n)$, significantly improving scalability.

M5 has $\Theta(n \log^2 n)$ complexity, as established using the extended Master Theorem and recursion-tree analysis. Its estimated processing cost fits the theoretical time window, but actual performance should be validated.

The report generator's function $f(n) = 3n^2 + 10n \log_2 n + 50$ is formally $\Theta(n^2)$ and is not $O(n)$.

Final decision: Redesign M1 and M4 as the highest priorities, retain M2 and M3 S2, and validate M5 through realistic profiling. These changes will improve ShopSphere's ability to handle one million products while keeping nightly processing within the required window, subject to real-world performance testing.